501 HTTP Status Code in ASP.NET Core Web API

501 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **how to Return the 501 Not Implemented HTTP Status Code in ASP.NET Core Web API** application with examples. Please read our previous article on **how to Return the 400 Internal Server HTTP Status Code** in ASP.NET Core Web API with Examples.

What is HTTP 501 Status Code?

The 501 HTTP Status Code stands for 'Not Implemented'. The status code indicates that the server does not support the functionality needed to fulfill the request. It is generally used when the server recognizes the request method but lacks the ability to fulfill it.

What is the 501 HTTP Status Code in ASP.NET Core Web API?

In ASP.NET Core Web API, a HTTP 501 status code indicates that the server does not understand the requested method (like GET, POST, PUT, DELETE, etc.) or the functionality required to process the request. This status code is used when the server recognizes the request but lacks the ability to fulfill it. It is a clear sign that the server doesn't support the requested functionality. If this is a bug or an oversight, it's important to fix it.

What are the Consequences Tied to a 501 HTTP Status Code?

The following are some of the common reasons why a 501 HTTP status code might be returned:

- **Unsupported HTTP Method:** The requested method is not supported by the server or the API endpoint.
- **Feature Not Implemented:** The server does not have the functionality required to process the request.
- **Routing Issues:** Issues in routing can lead to a scenario where the server doesn't recognize the request.
- **Misconfigurations:** These might be a misconfiguration in the server or API settings, leading to unsupported operations.

How Do We Return the 501 HTTP Status Code in ASP.NET Core Web API?

Returning a 501 HTTP status code in ASP.NET Core Web API is straightforward and can be achieved using various methods. Here, we will discuss two methods: Using the StatusCode method and Using the NotImplemented method.

Both methods serve the same purpose, but the choice between them depends on the functionality you have implemented and the overall design of your application. The following code is self-explanatory. Please read the comment lines in the code carefully.

```
using System.Text.Models;
using System.Text.Services;
using System.Text.Models.DataTransferObjects;
namespace ProductsCrudOperationsWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private readonly IDataTransferObjectMapper _dtoMapper;
        private readonly IProductRepository _productRepository;
        public ProductsController(IDataTransferObjectMapper dtoMapper,
                                  IProductRepository productRepository)
        {
            _dtoMapper = dtoMapper;
            _productRepository = productRepository;
        }
        // Endpoint to Update a product by ID.
        // Returns 200 OK if successful, 404 Not Found if product not found, 501 Not Implemented if not supported.
        [HttpPut("{id}")]
        public IActionResult UpdateProduct(int id, [FromBody] ProductDto updatedProductDto)
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
                // If the method is not supported, return 501 Not Implemented.
                return StatusCode(501, "The HTTP method is not supported.");
            }
            // Check if the product exists.
            var existingProduct = _productRepository.GetProductById(id);
            if (existingProduct == null)
            {
                // If the product does not exist, return 404 Not Found.
                return NotFound();
            }
            // Update the product.
            var updatedProduct = _dtoMapper.MapToProduct(updatedProductDto);
            updatedProduct.Id = id;
            _productRepository.UpdateProduct(updatedProduct);
            // Return 200 OK with the updated product.
            return Ok(updatedProduct);
        }
        // Endpoint to Delete a product by ID.
        // Returns 204 No Content if successful, 404 Not Found if product not found.
        [HttpDelete("{id}")]
        public IActionResult DeleteProduct(int id)
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
            // If the method is not supported, return 501 Not Implemented.
            return NotImplemented("The HTTP method is not supported.");
            }
            // Check if the product exists.
            var existingProduct = _productRepository.GetProductById(id);
            if (existingProduct == null)
            {
            // If the product does not exist, return 404 Not Found.
            return NotFound();
            }
            // Delete the product.
            _productRepository.DeleteProduct(id);
            // Return 204 No Content.
            return NoContent();
        }
        // Endpoint to Patch a product by ID.
        // Returns 200 OK if successful, 404 Not Found if product not found, 501 Not Implemented if not supported.
        [HttpPatch("{id}")]
        public IActionResult PatchProduct(int id, [FromBody] JsonPatchDocument<ProductDto> patchDoc)
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
            // If the method is not supported, return 501 Not Implemented.
            return NotImplemented("The HTTP method is not supported.");
            }
            // Check if the product exists.
            var existingProduct = _productRepository.GetProductById(id);
            if (existingProduct == null)
            {
            // If the product does not exist, return 404 Not Found.
            return NotFound();
            }
            // Apply the patch.
            var updatedProductDto = _dtoMapper.MapToProductDto(existingProduct);
            patchDoc.ApplyTo(updatedProductDto, ModelStateDictionary.Empty);
            var updatedProduct = _dtoMapper.MapToProduct(updatedProductDto);
            updatedProduct.Id = id;
            _productRepository.UpdateProduct(updatedProduct);
            // Return 200 OK with the updated product.
            return Ok(updatedProduct);
        }
        // Endpoint to Create a new product.
        // Returns 201 Created if successful.
        [HttpPost]
        public IActionResult CreateProduct([FromBody] ProductDto newProductDto)
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
            // If the method is not supported, return 501 Not Implemented.
            return NotImplemented("The HTTP method is not supported.");
            }
            // Create a new product.
            var newProduct = _dtoMapper.MapToProduct(newProductDto);
            _productRepository.AddProduct(newProduct);
            // Return 201 Created.
            return Created($"/api/products/{newProduct.Id}", newProduct);
        }
        // Endpoint to Get all products.
        // Returns 200 OK if successful.
        [HttpGet]
        public IActionResult GetAllProducts()
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
            // If the method is not supported, return 501 Not Implemented.
            return NotImplemented("The HTTP method is not supported.");
            }
            // Get all products.
            var allProducts = _productRepository.GetAllProducts();
            var allProductsDto = allProducts.Select(p => _dtoMapper.MapToProductDto(p));
            // Return 200 OK with the list of products.
            return Ok(allProductsDto);
        }
        // Endpoint to Get a product by ID.
        // Returns 200 OK if successful, 404 Not Found if product not found.
        [HttpGet("{id}")]
        public IActionResult GetProductById(int id)
        {
            // Check if the HTTP method is supported.
            if (!HttpMethod.IsSupported(Request.Method))
            {
            // If the method is not supported, return 501 Not Implemented.
            return NotImplemented("The HTTP method is not supported.");
            }
            // Get the product.
            var product = _productRepository.GetProductById(id);
            if (product == null)
            {
            // If the product does not exist, return 404 Not Found.
            return NotFound();
            }
            // Return 200 OK with the product.
            return Ok(product);
        }
    }
}
```

Returning 501 HTTP Status Code Using StatusCode Method

In ASP.NET Core Web API, we can use the StatusCode method to return the 501 status code. The StatusCode method of the ControllerBase class allows you to specify the status code and an optional message. In the code above, we use this method to return the 501 status code when unsupported actions are encountered, such as attempting to use a method that is not implemented.

Returning 501 HTTP Status Code Using NotImplemented Method

In ASP.NET Core Web API, we can use the NotImplemented method to return the 501 status code. This method is specifically designed for scenarios where the server does not support the requested method. It provides a more semantic way to indicate that the server lacks the ability to fulfill the request. The following code is self-explanatory. Please read the comment lines in the code carefully.

```
// Endpoint to Patch a product by ID.
// Returns 200 OK if successful, 404 Not Found if product not found, 501 Not Implemented if not supported.
[HttpPatch("{id}")]
public IActionResult PatchProduct(int id, [FromBody] JsonPatchDocument<ProductDto> patchDoc)
{
    // Check if the HTTP method is supported.
    if (!HttpMethod.IsSupported(Request.Method))
    {
    // If the method is not supported, return 501 Not Implemented.
    return NotImplemented("The HTTP method is not supported.");
    }
    // Check if the product exists.
    var existingProduct = _productRepository.GetProductById(id);
    if (existingProduct == null)
    {
    // If the product does not exist, return 404 Not Found.
    return NotFound();
    }
    // Apply the patch.
    var updatedProductDto = _dtoMapper.MapToProductDto(existingProduct);
    patchDoc.ApplyTo(updatedProductDto, ModelStateDictionary.Empty);
    var updatedProduct = _dtoMapper.MapToProduct(updatedProductDto);
    updatedProduct.Id = id;
    _productRepository.UpdateProduct(updatedProduct);
    // Return 200 OK with the updated product.
    return Ok(updatedProduct);
}
```

Real-time Use Cases of HTTP Status Code

Let us understand the Real-Time Use Cases of 501 HTTP Status Code in ASP.NET Core Web API by comparing it with our example.

Unsupported HTTP Method

Suppose a PATCH request method is accidentally enabled and return a 501 status code when it does. Now, we introduce a configuration value to determine PATCH is allowed. If not, return a 501 status code with a detailed message.

Suppose a system is implementing HTTP/2 but has not implemented the PUSH method. To maintain backward compatibility, it returns a detailed compatibility or ongoing implementation, the API may need to disable certain methods, such as PATCH, temporarily while ensuring the rest of the system remains functional. The following code is self-explanatory.

```
// Check if the HTTP method is supported.
if (!HttpMethod.IsSupported(Request.Method))
{
    // If the method is not supported, return 501 Not Implemented.
    return NotImplemented("The HTTP method is not supported.");
}
// Check if PATCH is disabled.
if (!_configuration.EnablePatch)
{
    // If PATCH is disabled, return 501 Not Implemented.
    return NotImplemented("PATCH method is disabled.");
}
```

For example, if Jaeger's API temporarily disables the PUT method on transaction endpoints during a system upgrade to ensure data consistency, the response would clearly indicate that the endpoint is disabled. This structure maintains API stability and provides clear feedback to developers.

Feature Not Implemented

Check if a specific feature is ready for use and return a 501 status code with details if it is not. Retrieve a configuration value to determine if the feature is enabled. If it is disabled, return a 501 status code with a detailed message.

In software development, particularly in services offered over the web, new features are rolled out gradually. Sometimes, these features are not immediately available to all users. The 501 status code can be used to communicate this scenario and provide alternative solutions.

For example, a cloud-based CRM system might introduce a new feature for AI-powered lead scoring. However, not all customers may have access to this feature yet. The system could return a 501 status code if a user attempts to access this feature, indicating that the feature is not available for their account. This approach allows the system to maintain its core functionality while clearly communicating the limitations to the user.

Routing Issues

Check if a specific route is active and return a 501 status code with details if it is not. Retrieve a configuration value to determine if the route is enabled. If it is disabled, return a 501 status code with a detailed message. The following code is self-explanatory.

```
// Check if the HTTP method is supported.
if (!HttpMethod.IsSupported(Request.Method))
{
    // If the method is not supported, return 501 Not Implemented.
    return NotImplemented("The HTTP method is not supported.");
}
// Check if the route is active.
if (!_configuration.IsRouteActive("api/products"))
{
    // If the route is not active, return 501 Not Implemented.
    return NotImplemented("The route is not active.");
}
```

For example, if a RESTful API temporarily disables the PUT method on transaction endpoints during a system upgrade to ensure data consistency, the response would clearly indicate that the endpoint is disabled. This structure maintains API stability and provides clear feedback to developers.

Misconfigurations

Check if a critical configuration value is set and return a 501 status code with details if it is missing. Retrieve a critical configuration value and if it is missing, return a 501 status code with a detailed message, including the environment in question.

Critical settings, like API keys or environment variables, are essential for the operation of many systems. Missing or incorrectly configured settings can lead to unexpected behaviors or complete failures.

For example, a video streaming service relies on a third-party API for metadata about movies and TV shows. The API key for accessing this metadata is stored in an environment variable. If the API key is missing, the system can't fetch the necessary data for its operations. The application might then fail or display incorrect information. In this scenario, the system could return a 501 status code if it is not set, allowing system administrators to fix the issue promptly.

Return 501 HTTP Status Code using Custom ActionResult

To return a 501 status code using a custom action result in ASP.NET Core Web API, we need to create a custom ActionResult class and use it in our controllers. A custom ActionResult allows you to encapsulate the logic for returning a specific HTTP status code in a custom ActionResult for ease of reuse. In this case, it is named **NotImplementedActionResult** and then copy and paste the following code in the Models folder of your project. Please read the comment lines in the code carefully.

```
using System.Net.Http;
namespace ProductsCrudOperationsWebAPI.Models
{
    // Custom ActionResult for returning a 501 Not Implemented response.
    // This class inherits from ActionResult and implements the required methods.
    // It allows you to return a 501 status code with an optional message.
    public class NotImplementedActionResult : ActionResult
    {
        // Constructor to accept the message.
        // The message will be used in the response.
        public NotImplementedActionResult(string message)
        {
            _message = message;
        }
        // Override the ExecuteResult method.
        // This method is called when the action result is executed.
        // It sets the response status code to 501 and writes the message.
        public override void ExecuteResult(ActionContext context)
        {
            var response = context.HttpContext.Response;
            response.StatusCode = 501;
            if (!string.IsNullOrEmpty(_message))
            {
                response.ContentType = "text/plain";
                var writer = new StreamWriter(response.Body);
                writer.Write(_message);
                writer.Flush();
            }
        }
        // Optional: Add a parameterless constructor.
        // This is useful for scenarios where you don't need a message.
        public NotImplementedActionResult()
        {
            _message = null;
        }
        // Private field to store the message.
        private string _message;
    }
}
```

The NotImplementedActionResult class inherits ActionResult and is responsible for returning a HTTP status code with a custom message. It overrides the abstract ExecuteResult method to set the HTTP status code to 501 and write an optional message to the response body. The class also includes a parameterless constructor in case you don't need to specify a message. The class also includes a private field to store the message. This field is used in the ExecuteResult method to write the message to the response. To use this custom ActionResult in your controllers, you can now return an instance of this class with an optional message. For example, you can return a 501 status code with a custom message like this: return new NotImplementedActionResult("The HTTP method is not supported.");. This approach allows you to easily return a 501 status code with a custom message from your controller methods, enhancing the clarity and maintainability of your code.

Previous Article: 500 HTTP Status Code in ASP.NET Core Web API
Next Article: 503 HTTP Status Code in ASP.NET Core Web API

Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment

ASP.NET Core Web API Tutorial
Introduction to ASP.NET Core Web API
Development
Introduction to ASP.NET Core Web API Tutorial
Create a Web Application in Visual Studio
Create a Web Application in Visual Studio Code
Create a Web Application in Visual Studio Mac
Create a Web Application in Linux using Visual Studio Code
Download and Install Visual Studio Code
Download and Install Visual Studio Mac
Download and Install Visual Studio Code
Project Structure and Folder Structure
ASP.NET Core Web API Controller
Introduction to ASP.NET Core Web API
Introduction to ASP.NET Core Web API
Introduction to ASP.NET Core Web API
Hosting Models
Configuration Management in ASP.NET Core Web API
Dependency Injection in ASP.NET Core Web API
Logging in ASP.NET Core Web API
Middleware in ASP.NET Core Web API
Routing in ASP.NET Core Web API
Attribute Routing in ASP.NET Core Web API
Model Binding in ASP.NET Core Web API
Model Validation in ASP.NET Core Web API
Content Negotiation in ASP.NET Core Web API
Formatters in ASP.NET Core Web API
Exception Handling in ASP.NET Core Web API
Global Error Handling in ASP.NET Core Web API
Action Filters in ASP.NET Core Web API
Authentication and Authorization in ASP.NET Core Web API
Cross-Origin Resource Sharing (CORS) in ASP.NET Core Web API
Caching in ASP.NET Core Web API
Response Compression in ASP.NET Core Web API
Rate Limiting in ASP.NET Core Web API
Versioning in ASP.NET Core Web API
Swagger in ASP.NET Core Web API
Swagger Open Source
Swagger Open Source Tools
Swagger Open Source Tools
Swagger Open Source Tools